

JSP 模式&EL&JSTL

今日内容介绍

- ◆ 案例：重写商品信息展示

今日内容学习目标

- ◆ 阐述 MVC 设计模式思想
- ◆ 绘制三层体系架构执行流程图
- ◆ 会使用 EL 从指定作用域获得数据
- ◆ 会使用 JSTL 的 forEach 遍历数据
- ◆ 会使用 JSTL 的 if 语句进行数据过滤

第1章 案例：商品信息展示

1.1 需求

- 重写商品信息展示

1.2 相关知识点

1.2.1 EL 表达式的概述

在 JSP 开发中，为了获取 Servlet 域对象中存储的数据，经常需要书写很多 Java 代码，这样的做法会使 JSP 页面混乱，难以维护，为此，在 JSP2.0 规范中提供了 EL 表达式。EL 是 Expression Language 的缩写，它是一种简单的数据访问语言。本节将针对 EL 表达式进行详细的讲解。

1.2.1.1 什么是 EL

EL (Expression Language) 目的：为了使 JSP 写起来更加简单。表达式语言的灵感来自于 ECMAScript 和 XPath 表达式语言，它提供了在 JSP 中简化表达式的方法，让 Jsp 的代码更加简化。

1.2.1.2 EL 的语法

由于 EL 可以简化 JSP 页面的书写，因此，在 JSP 的学习中，掌握 EL 是相当重要的。要使用 EL 表达式，首先要学习它的语法。EL 表达式的语法非常简单，都是以“\${”符号开始，以“}”符号结束的，具体格式如下：

```
${表达式}
```

1.2.1.3 EL 的使用：内置对象

分类	内置对象名称	描述
作用域	pageScope	page 作用域
	requestScope	request 作用域
	sessionScope	session 作用域
	applicationScope	application 作用域
请求参数	param	获得一个参数
	paramValues	获得一组参数
请求头	header	获得一个请求头
	headerValues	获得一组请求头
JSP 上下文对象	pageContext	
全局初始化参数	initParam	
cookie	cookie	

● 获得指定作用域的数据

```
<%--初始化数据--%>
<%
    pageContext.setAttribute("name", "pValue");
    request.setAttribute("name", "rValue");
    session.setAttribute("name", "sValue");
    application.setAttribute("name", "aValue");
%>
<%--使用 JSP 脚本获得--%>
<%=pageContext.getAttribute("name") %> <!-- 如果没找到 返回 null -->
<%=request.getAttribute("name") %>
<%=session.getAttribute("name") %>
<%=application.getAttribute("name") %>
```

```
<%--获得指定作用域的数据--%>
${ pageScope.name } <!-- 返回的是"" -->
${ requestScope.name }
${ sessionScope.name }
${ applicationScope.name }

<%--依次获得数据--%>
${ name } <!-- 底层使用 pageContext.findAttribute("name") ,依次从 page、request、
session、application 获得数据,如果都没有返回 null --%>
```

● 请求参数

```
//请求路径: /day18/a_el.jsp?username=jack&hobby=xxx&hobby=yyy
<%--
    param.xxx      对应 request.getParameter("xxx");
    paramValue.xxx 对应 request.getParameterValues("xxx");
--%>
${param.username} <br/>
${param.hobby} <br/> <%--获得第一个参数--%>
${paramValues.hobby} <br/> <%--获得一组数据,使用数组 --%>
${paramValues.hobby[1]} <br/> <%--如果是数组,可以使用下标获得 --%>
```

● 请求头

```
<%--
    header.xxx      对应 request.getHeader("xxx");
    headerValues.xxx 对应 request.getHeaders("xxx");
--%>
${header.accept} <br/>
\${header.accept-Encoding} <br/> <%--非法的,有异常,“-”被解析成减号。使用"/"进行单个 el
表达式转义 --%>
${header['accept-Encoding']} <br/>
${headerValues['accept-Encoding'][0]} <br/>
```

● pageContext

```
<%--
    pageContext 对应 pageContext 对象
--%>

jsp: <%= ((HttpServletRequest)pageContext.getRequest()).getContextPath() %> <br/>
el : ${pageContext.request.contextPath} <br/>
```

● 全局初始化参数

```
<%--
    web.xml 配置
```

```
<context-param>
    <param-name>person</param-name>
    <param-value>白跃</param-value>
</context-param>

initParam 对应 servletContext.getInitParameter("person");
--%>
${initParam.person}
```

- cookie

```
<%--
    cookie 没有对应 api。底层使用 request.getCookies() 获得所有 cookie，然后遍历并存放 Map
    中 (Map<name,obj>)
    --%>

    ${cookie} <br/>          <!--使用 Map 存所有 cookie，Map<名称, 对象> --%>
    ${cookie.company}      <!--map.key 获得对应 value --%>
    ${cookie.company.name} <!--通过 javabean 属性获得属性，获得 cookie 的名称，对应方法
    getName()--%>
    ${cookie.company.value} <!--获得 cookie 的值，对应方法 getValue() --%>
```

- .和[]的区别.

- 1) []用于有下标的数据(数组,list 集合) .用于有属性的数据(map, 对象)
- 2) 如果属性名中包含有特殊的字符.必须使用[]

1.2.1.4 EL 的使用：获得数据

- 自定义数据，必须存放在作用域

```
<%
    String str = "传智播客";
%>

${str} <!--无法获得，str 不在域中 --%>
```

- 获得字符串

```
<%
    String s = "黑马程序员";
    //将输出存放到作用域
    pageContext.setAttribute("sss", s);
%>
${sss} <br/>
${pageScope.sss} <br/>
```

- 获得数组

```
<%
    String[] arr = {"A","B","C"};
```

```
        pageContext.setAttribute("arr", arr, PageContext.REQUEST_SCOPE);
    %>
    ${arr} <br/>
    ${arr[1]} <br/>
```

- 获得 List 数据

```
<%
    //模拟数据
    List<String> list = new ArrayList<String>();
    list.add("zhangsan");
    list.add("lisi");
    list.add("wangwu");
    pageContext.setAttribute("list", list);
%>
${list} <br/> <%-- 输出格式: [ , , ] --%>
${list[2]} <br/>
```

- 获得 Map<String,String>数据

```
<%
    //模拟数据
    Map<String,String> map = new HashMap<String,String>();
    map.put("li", "小李");
    map.put("guo", "小郭");
    map.put("wang", "老王");
    pageContext.setAttribute("map", map);
%>

${map} <br/> <%-- 输出结果: { k=v ,...} --%>
${map.li} <br/> <%-- 通过 key 获得数据--%>
```

- 获得 Map<String,JavaBean>数据

```
<%
    Map<String,User> map2 = new HashMap<String,User>();
    map2.put("u1", new User("u001","jack",18));
    map2.put("u2", new User("u002","rose",21));
    map2.put("3", new User("u003","tom",25));
    pageContext.setAttribute("map2", map2);
%>

${map2} <br/>
${map2.u2} <br/> <%--通过 key 获得数据 --%>
${map2.u2.userName} <%--通过 javabean 属性获得数据 --%>
${map2['3'].userName} <%--可以通过字符串'3'获得数据, 注意: Map.key 类型为 Integer, 将不
```


能获得数据 --%>

1.2.1.5 EL 的使用：运算符

- 模拟数据

```
<%
    pageContext.setAttribute("n1", "10");
    pageContext.setAttribute("n2", "20");
    pageContext.setAttribute("n3", "30");
    pageContext.setAttribute("n4", "40");
%>
```

- 算术运算符

算术运算符	说明	范例	结果
+	加	<code>\${1+1}</code>	2
-	减	<code>\${2-1}</code>	1
*	乘	<code>\${1*1}</code>	1
/或div	除	<code>\${5 div 2}</code>	2.5
%或mod	取余	<code>\${5 mod 2}</code>	1

```
${ n1 + n2 + n3 }
${1+1} <br/>
${"1"+1} <br/> <!--将字符串转换成数字，然后进行计算 --%>
${'1'+1} <br/> <!--el 没有字符 --%>
\${'a'+1} <br/> <!-- 将 a 转换数字，异常 --%>
${a + 1} <br/> <!--a 将从作用域获得数据，如果没有 --%>
```

- 逻辑运算符

关系运算符	说明	范例	结果
<code>==</code> 或 <code>eq</code>	等于(<u>equ</u> al)	<code>\${1 eq 1}</code>	true
<code>!=</code> 或 <code>ne</code>	不等于(not equal)	<code>\${1 != 1}</code>	false
<code><</code> 或 <code>lt</code>	小于(Less than)	<code>\${1 lt 2}</code>	true
<code><=</code> 或 <code>le</code>	小于等于(Less than or equal)	<code>\${1 <= 1}</code>	true
<code>></code> 或 <code>gt</code>	大于(Greater than)	<code>\${1 > 2}</code>	false
<code>>=</code> 或 <code>ge</code>	大于等于(Greater than or equal)	<code>\${1 >= 1}</code>	true

```
{ n1 < n2 } - { n1 lt n2 } <!-- less than --><br/>
{ n1 > n2 } - { n1 gt n2 } <!-- great than --><br/>
{ n1 <= n2 } - { n1 le n2 } <!-- less equal --><br/>
{ n1 >= n2 } - { n1 ge n2 } <!-- great equal --><br/>
{ n1 == n2 } - { n1 eq n2 } <!-- equal --><br/>
```

- 关系运算符

逻辑运算符	说明	范例	结果
&& 或 and	交集(与)	{A and B}	true / false
或 or	并集(或)	{A B }	true / false
! 或 not	非	{not A}	true / false

```
{ n1<n2 && n3 < n4 } - { n1<n2 and n3 < n4 }<br/>
{ n1<n2 || n3 < n4 } - { n1<n2 or n3 < n4 }<br/>
{ !(n1 < n2) } - { not(n1<n2) }
```

- 三元运算符

```
{ n1 < n2 ? "正确":"错误" }
```

- empty 运算符

```
<%--
    1.对象是否为 null
    2.字符串是否为""
    3.集合是否为 0
--%>

{ user == null } - { empty user }
{ user != null } - { not empty user }
```

1.2.2 JSTL

1.2.2.1 什么是 JSTL

JSTL

JSTL (JSP Standard Tag Library, JSP标准标签库)是一个不断完善的开放源代码的JSP标签库,是由apache的jakarta小组来维护的。JSTL只能运行在支持JSP1.2和Servlet2.3规范的容器上,如tomcat 4.x。在JSP 2.0中也是作为标准支持的。

JSTL 1.0 发布于 2002 年 6 月,由四个定制标记库 (core、format、xml 和 sql) 和一对通用标记库验证器 (ScriptFreeTLV 和 PermittedTaglibsTLV) 组成。core 标记库提供了定制操作,通过限制了作用域的变量管理数据,以及执行页面内容的迭代和条件操作。它还提供了用来生成和操作 URL 的标记。顾名思义,format 标记库定义了用来格式化数据 (尤其是数字和日期) 的操作。它还支持使用本地化资源来进行 JSP 页面的国际化。xml 库包含一些标记,这些标记用来操作通过 XML 表示的数据,而 sql 库定义了用来查询关系数据库的操作。

如果要使用JSTL,则必须将jstl.jar和 standard.jar文件放到classpath中,如果你还需要使用XML processing及Database access (SQL)标签,还要将相关JAR文件放到classpath中,这些JAR文件全部存在于下载回来的zip文件中。

从 JSP1.1 规范开始, JSP 就支持使用自定义标签,使用自定义标签大大降低了 JSP 页面的复杂度,同时增强了代码的重用性。为此,许多 Web 应用厂商都定制了自身应用的标签库,然而同一功能的标签由不同的 Web 应用厂商制定可能是不同的,这就导致市面上出现了很多功能相同的标签,令网页制作者无从选择,为了解决这个问题, Sun 公司制定了一套标准标签库 (JavaServer Pages Standard Tag Library), 简称 JSTL。

JSTL 虽然被称为标准标签库,而实际上这个标签库是由 5 个不同功能的标签库共同组成的。在 JSTL1.1 规范中,为这 5 个标签库分别指定了不同的 URI 以及建议使用的前缀,如表 1-7 所示。

表1-1 JSTL 包含的标签库

标签库	标签库的 URI	前缀
Core	http://java.sun.com/jsp/jstl/core	c
l18N	http://java.sun.com/jsp/jstl/fmt	fmt
SQL	http://java.sun.com/jsp/jstl/sql	sql
XML	http://java.sun.com/jsp/jstl/xml	x
Functions	http://java.sun.com/jsp/jstl/functions	fn

1.2.2.2 JSTL 的安装和测试

通过 7.3.1 小节的学习,了解了 JSTL 的基本知识,要想在 JSP 页面中使用 JSTL,首先需要安装 JSTL。接下来,分步骤演示 JSTL 的安装和测试,具体如下:

1. 下载 JSTL 包

从 Apache 的网站下载 JSTL 的 JAR 包。进入

“<http://archive.apache.org/dist/jakarta/taglibs/standard/binaries/>” 网址下载 JSTL 的安装包 jakarta-taglibs-standard-1.1.2.zip,然后将下载好的 JSTL 安装包进行解压,此时,在 lib 目录下可以看到两个 JAR 文件,分别为 jstl.jar 和 standard.jar。其中, jstl.jar 文件包含 JSTL 规范中定义的接口和相关类, standard.jar 文件包含用于实现 JSTL 的.class 文件以及 JSTL 中 5 个标签库描述符文件(TLD)。

2. 安装 JSTL

将 `jstl.jar` 和 `standard.jar` 这两个文件复制到 `day18` 项目的 `lib` 目录下，如图 1-16 所示。

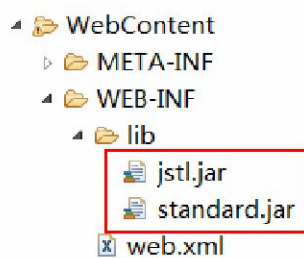


图1-1 导入 `jstl.jar` 和 `standard.jar` 文件

从图 1-8 可以看出，`jstl.jar` 和 `standard.jar` 这两个文件已经被导入到 `day18` 项目的 `lib` 文件夹中，这个过程就相当于在 `day18` 项目中安装 JSTL，安装完 JSTL 后，就可以在 JSP 文件中使用 JSTL 标签库。

3. 测试 JSTL

JSTL 安装完成后，就需要测试 JSTL 安装是否成功。由于在测试的时候使用的是 `<c:out>` 标签，因此，需要使用 `taglib` 指令导入 Core 标签库，具体代码如下：

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
```

在上述代码中，`taglib` 指令的 `uri` 属性用于指定引入标签库描述符文件的 URI，`prefix` 属性用于指定引入标签库描述符文件的前缀，在 JSP 文件中使用这个标签库中的某个标签时，都需要使用这个前缀。

接下来编写一个简单的 JSP 文件 `test.jsp`，使用 `taglib` 指令引入 Core 标签库，在该文件中使用 `<c:out>` 标签，如文件 1-12 所示。

文件1-1 test.jsp

```
<%@ page language="java" contentType="text/html; charset=utf-8"
    pageEncoding="utf-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<html>
<head></head>
<body>
    <c:out value="Hello World!"></c:out>
</body>
</html>
```

启动 Tomcat 服务器，打开浏览器，在地址栏中输入地址“`http://localhost:8080/chapter07/test.jsp`”访问 `test.jsp` 页面，此时，浏览器窗口中显示的结果如图 1-17 所示。

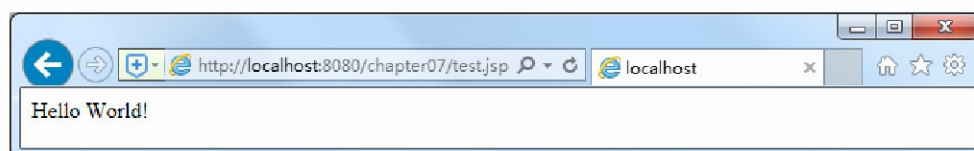


图1-2 test.jsp

从图 1-17 可以看出，使用浏览器访问 `test.jsp` 页面时，输出了“Hello World!”，由此可见，JSTL 标签库安装成功了。

1.2.2.3 Core 标签库：if 标签

在程序开发中，经常需要使用 `if` 语句进行条件判断，如果要在 JSP 页面中进行条件判断，就需

要使用 Core 标签库提供的<c:if>标签，该标签专门用于完成 JSP 页面中的条件判断，它有两种语法格式，具体如下：

语法 1：没有标签体的情况，将结果存放到指定的作用域（不常用）

```
<c:if test="testCondition" var="result"
[scope="{page|request|session|application}"]/>
```

语法 2：有标签体的情况，在标签体中指定要输出的内容

```
<c:if test="testCondition" >
    body content
</c:if>
```

在上述语法格式中，可以看到<c:if>标签有三个属性，接下来将针对这三个属性进行讲解，具体如下：

- test 属性用于设置逻辑表达式；
- var 属性用于指定逻辑表达式中变量的名字；
- scope 属性用于指定 var 变量的作用范围，默认值为 page。如果属性 test 的计算结果为 true，那么标签体将被执行，否则标签体不会被执行。

通过前面的讲解，我们对<c:if>标签有了一个简单的认识，接下来通过一个具体的案例来演示如何在 JSP 页面中使用<c:if>标签。

在 WebContent 目录下创建一个名为 c_if.jsp 的文件，代码如文件 1-15 所示。

文件1-2 c_if.jsp

```
<%@ page language="java" contentType="text/html; charset=utf-8"
pageEncoding="utf-8" import="java.util.*"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<html>
<head></head>
<body>
    <c:set value="1" var="visitCount" property="visitCount" />
    <c:if test="${visitCount==1}">
        This is you first visit. Welcome to the site!
    </c:if>
</body>
</html>
```

启动 Tomcat 服务器，在浏览器地址栏中输入地址“http://localhost:8080/chapter07/c_if.jsp”访问 c_if.jsp 页面，此时，浏览器窗口中显示的结果如图 1-18 所示。

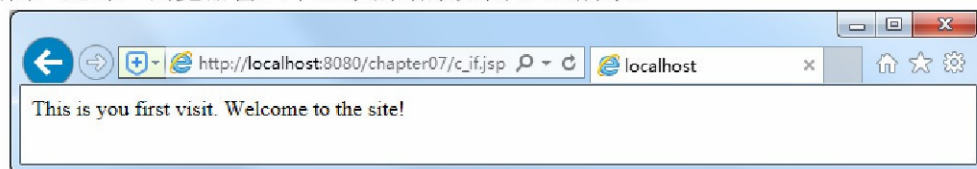


图1-3 c_if.jsp

从图 1-18 可以看出，浏览器窗口中显示了<c:if>标签体中的内容。这是因为在文件 1-15 中使用了<c:if>标签，当执行到<c:if>标签时会通过 test 属性来判断表达式\${visitCount==1}是否为 true，如果为 true 就输出标签体中的内容，否则输出空字符串。由于使用了<c:set>标签将 visitCount 的值设置为 1，因此，表达式\${visitCount==1}的结果为 true，便会输出<c:if>标签体中的内容。

1.2.2.4 Core 标签库：forEach 标签

在 JSP 页面中,经常需要对集合对象进行循环迭代操作,为此,Core 标签库提供了一个<c:forEach>标签,该标签专门用于迭代集合对象中的元素,如 Set、List、Map、数组等,并且能重复执行标签体中的内容,它有两种语法格式,具体如下:

语法 1: 迭代包含多个对象的集合

```
<c:forEach [var="varName"] items="collection" [varStatus="varStatusName"]
[begin="begin"] [end="end"] [step="step"]>
    body content
</c:forEach>
```

语法 2: 迭代指定范围内的集合

```
<c:forEach [var="varName"] [varStatus="varStatusName"] begin="begin"
end="end" [step="step"]>
    body content
</c:forEach>
```

在上述语法格式中,可以看到<c:forEach>标签有多个属性。接下来针对这些属性进行讲解,具体如下:

- var 属性用于指将当前迭代到的元素保存到 page 域中的名称;
- items 属性用于指定将要迭代的集合对象;
- varStatus 用于指定当前迭代状态信息的对象保存到 page 域中的名称;
- begin 属性用于指定从集合中第几个元素开始进行迭代,begin 的索引值从 0 开始,如果没有指定 items 属性,就从 begin 指定的值开始迭代,直到迭代结束为止;
- step 属性用于指定迭代的步长,即迭代因子的增量。

<c:forEach>标签在程序开发中经常会被用到,因此熟练掌握<c:forEach>标签是很有必要的,接下来,通过几个具体的案例来学习<c:forEach>标签的使用。

分别使用<c:forEach>标签迭代数组和 Map 集合,首先需要在数组和 Map 集合中添加几个元素,然后将数组赋值给<c:forEach>标签的 items 属性,而 Map 集合对象同样赋值给<c:forEach>标签的 items 属性,之后使用 getKey()和 getValue()方法就可以获取到 Map 集合中的键和值,如文件 1-17 所示。

文件1-3 c_forEach1.jsp

```
<%@ page language="java" contentType="text/html; charset=utf-8"
    pageEncoding="utf-8" import="java.util.*"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<html>
<head></head>
<body>
    <%
        String[] fruits = { "apple", "orange", "grape", "banana" };
    %>
    String 数组中的元素:
    <br />
    <c:forEach var="name" items="%=fruits%>"
        ${name}<br />
    </c:forEach>
```



```

<%
    Map userMap = new HashMap();
    userMap.put("Tom", "123");
    userMap.put("Make", "123");
    userMap.put("Lina", "123");
%>
<hr />
HashMap 集合中的元素:
<br />
<c:forEach var="entry" items="%=userMap%">
    ${entry.key}&nbsp;${entry.value}<br />
</c:forEach>
</body>
</html>

```

启动 Tomcat 服务器，在浏览器在地址栏中输入地址“http://localhost:8080/chapter07/c_foreach1.jsp”访问 c_foreach1.jsp 页面，此时，浏览器窗口中的显示结果如图 1-19 所示。

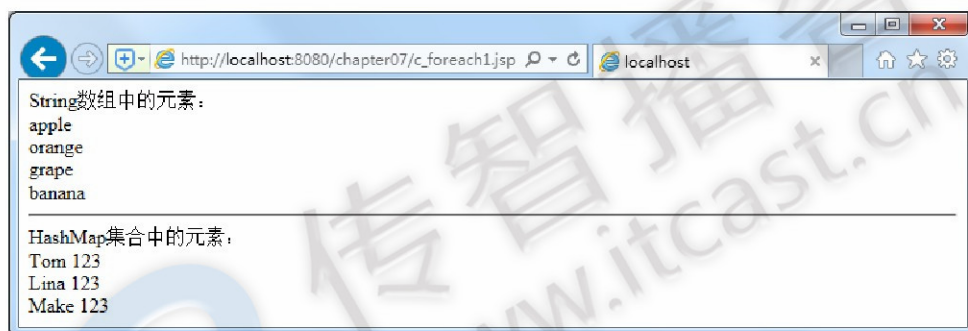


图1-4 c_foreach1.jsp

从图 1-19 可以看出，在 String 数组中存入的元素 apple、orange、grape 和 banana 全部被打印出来了，因此可以说明使用<c:forEach>标签可以迭代数组中的元素。Map 集合中存入的用户名和密码全部被打印出来了。在使用<c:forEach>标签时，只需将 userMap 集合对象赋值给 items 属性，之后通过 entry 变量就可以获取到集合中的键和值。

<c:forEach>标签的 begin、end 和 step 属性分别用于指定循环的起始索引、结束索引和步长。使用这些属性可以迭代集合对象中某一范围内的元素。

在项目的 WebContent 目录下，创建一个名为 c_foreach2.jsp 的文件，该文件中使用了<c:forEach>标签的 begin、end 和 step 属性，代码如文件 1-14 所示。

文件1-4 c_foreach2.jsp

```

<%@ page language="java" contentType="text/html; charset=utf-8"
    pageEncoding="utf-8" import="java.util.*"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<html>
<head></head>
<body>
    colorsList 集合（指定迭代范围和步长）<br />
    <%
        List colorsList=new ArrayList();

```

```

        colorsList.add("red");
        colorsList.add("yellow");
        colorsList.add("blue");
        colorsList.add("green");
        colorsList.add("black");
    %>
    <c:forEach var="color" items="<%=colorsList%>" begin="1"
    end="3" step="2">
        ${color}&nbsp;
    </c:forEach>
</body>
</html>

```

启动 Tomcat 服务器,在浏览器地址栏中输入地址“http://localhost:8080/chapter07/c_foreach2.jsp”访问 `c_foreach2.jsp` 页面,此时,浏览器窗口中的显示结果如图 1-20 所示。

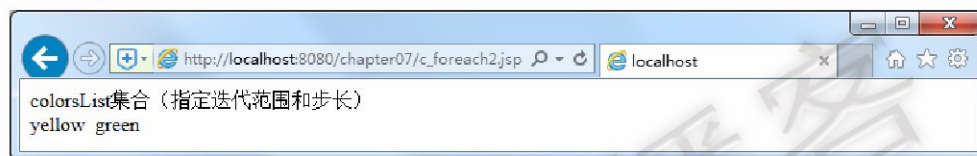


图1-5 c_foreach2.jsp

从图 1-20 可以看出,浏览器窗口中显示了 `colorsList` 集合中的 `yellow` 和 `green` 两个元素,只显示这两个元素的原因是,在使用 `<c:forEach>` 标签迭代 List 集合时,指定了迭代的起始索引为 1,当迭代集合时首先会输出 `yellow` 元素,由于在 `<c:forEach>` 标签中指定了步长为 2,并且指定了迭代的结束索引为 3,因此,还会输出集合中的 `green` 元素,其他的元素不会再输出。

`<c:forEach>` 标签的 `varStatus` 属性用于设置一个 `javax.servlet.jsp.jstl.core.LoopTagStatus` 类型的变量,这个变量包含了从集合中取出元素的状态信息。使用 `<c:forEach>` 标签的 `varStatus` 属性可以获取以下信息:

- **count**: 表示元素在集合中的序号,从 1 开始计数;
- **index**: 表示当前元素在集合中的索引,从 0 开始计数;
- **first**: 表示当前是否为集合中的第一个元素;
- **last**: 表示当前是否为集合中的最后一个元素;

通过上面的讲解,读者对 `<c:forEach>` 标签的 `varStatus` 属性已经有了基本的了解,接下来通过一个具体的案例来演示如何使用 `<c:forEach>` 标签的 `varStatus` 属性获取集合中元素的状态信息

在项目的 `WebContent` 目录下创建一个名为 `c_foreach3.jsp` 的文件,代码如文件 1-15 所示。

文件1-5 c_foreach3.jsp

```

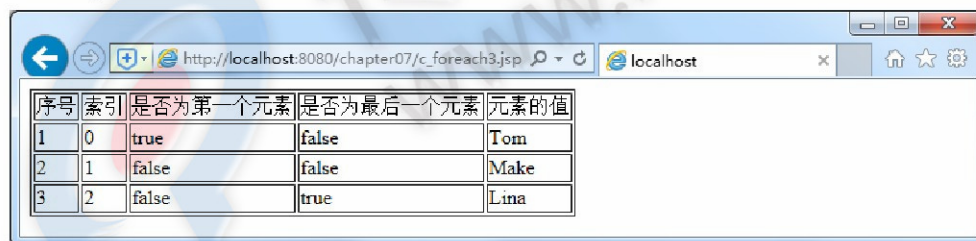
<%@ page language="java" contentType="text/html; charset=utf-8"
pageEncoding="utf-8" import="java.util.*"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<html>
<head></head>
<body style="text-align: center;">
    <%
        List userList = new ArrayList();
        userList.add("Tom");
        userList.add("Make");
    %>

```



```
        userList.add("Lina");
    }
    <table border="1">
        <tr>
            <td>序号</td>
            <td>索引</td>
            <td>是否为第一个元素</td>
            <td>是否为最后一个元素</td>
            <td>元素的值</td>
        </tr>
        <c:forEach var="name" items="<%=userList%>" varStatus="status">
            <tr>
                <td>${status.count}</td>
                <td>${status.index}</td>
                <td>${status.first}</td>
                <td>${status.last}</td>
                <td>${name}</td>
            </tr>
        </c:forEach>
    </table>
</body>
</html>
```

启动 Tomcat 服务器,在浏览器地址栏中输入地址“http://localhost:8080/chapter07/c_foreach3.jsp”访问 `c_foreach3.jsp` 页面,此时,浏览器窗口中的显示结果如图 1-21 所示。



序号	索引	是否为第一个元素	是否为最后一个元素	元素的值
1	0	true	false	Tom
2	1	false	false	Make
3	2	false	true	Lina

图1-6 c_foreach3.jsp

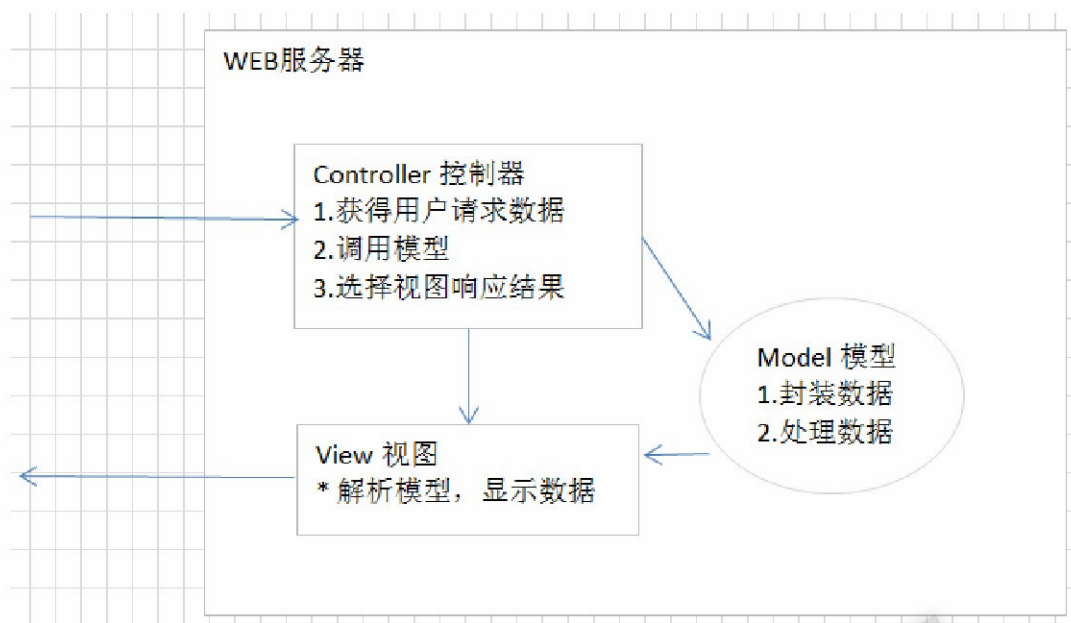
从图 1-21 可以看出,使用 `<c:forEach>` 标签迭代集合中的元素时,可以通过 `varStatus` 属性获取集合中元素的序号和索引,而且还可以判断集合中的元素是否为第一个元素以及最后一个元素。因此可以说明使用该属性可以很方便的获取集合中元素的状态信息。

1.2.3 MVC 设计模式

MVC 设计模式: Model-View-Controller 简写。

MVC 是软件工程中的一种软件架构模式,它是一种分离业务逻辑与显示界面的设计方法。它把软件系统分为三个基本部分:模型 (Model)、视图 (View) 和控制器 (Controller)。

- 控制器 Controller: 对请求进行处理,负责请求转发;
- 视图 View: 界面设计人员进行图形界面设计;
- 模型 Model: 编写程序应用的功能 (实现算法等等)、数据库管理;



MVC 可对程序的后期维护和扩展提供了方便，并且使程序某些部分的重用提供了方便。而且 MVC 也使程序简化，更加直观。

注意，MVC 不是 Java 的特有的，几乎现在所有 B/S 结构的软件都采用了 MVC 设计模式。

1.2.4 JSP 开发模式

当 SUN 公司推出 JSP 后，同时也提供相应的开发模式，JavaWeb 经历了 JSP Model1 第一代，JSPModel1 第二代，JSP Model 2 三个时期。

1.2.4.1 JSP Model1 第一代

JSP Model1 是 JavaWeb 早期的模型，它适合小型 Web 项目，开发成本低！Model1 第一代时期，服务器端只有 JSP 页面，所有的操作都在 JSP 页面中，连访问数据库的 API 也在 JSP 页面中完成。也就是说，所有的东西都耦合在一起，对后期的维护和扩展极为不利。

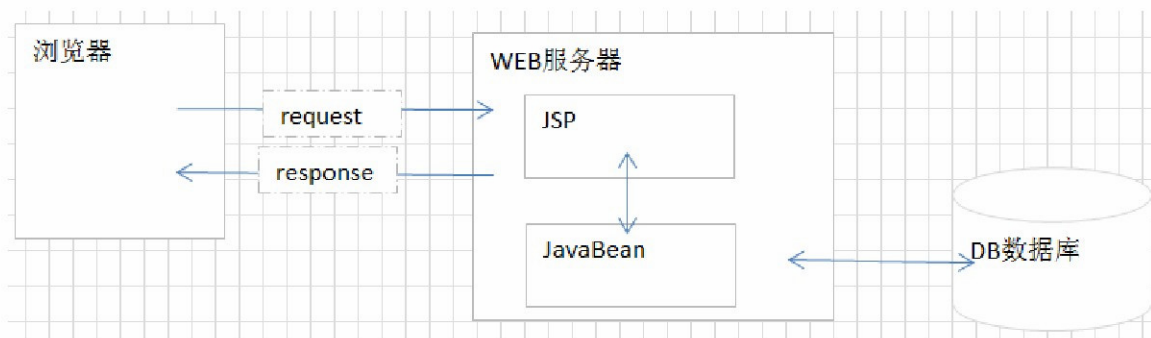
（高内聚低耦合：）



1.2.4.2 JSP Model1 第二代

JSP Model1 第二代有所改进，把业务逻辑的内容放到了 JavaBean 中，而 JSP 页面负责显示以及

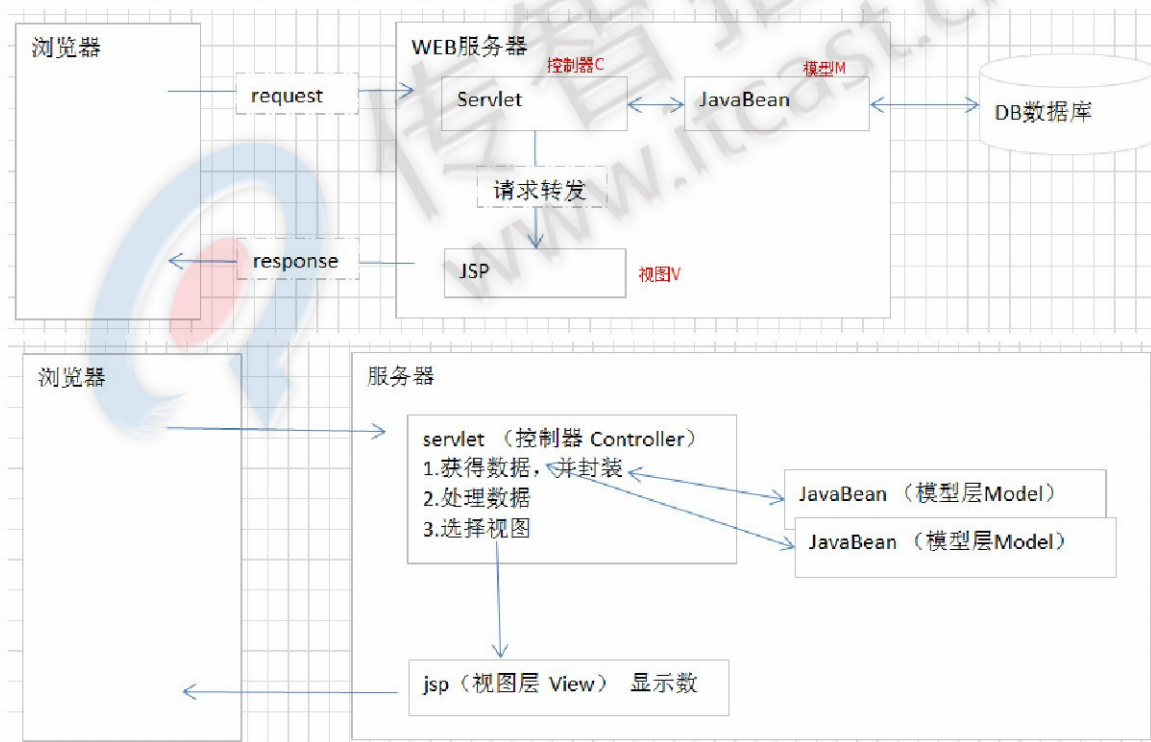
请求调度的工作。虽然第二代比第一代好了些，但还让 JSP 做了过多的工作，JSP 中把视图工作和请求调度（控制器）的工作耦合在一起了。



1.2.4.3 JSP Model 2

Model2 使用到的技术有：Servlet、JSP、JavaBean。Model2 是 MVC 设计模式在 Java 语言的具体体现。

- JSP：视图层，用来与用户打交道。负责接收传来的数据，以及显示数据给用户；
- Servlet：控制层，负责找到合适的模型对象来处理业务逻辑，转发到合适的视图；
- JavaBean：模型层，完成具体的业务工作，例如：转账等。

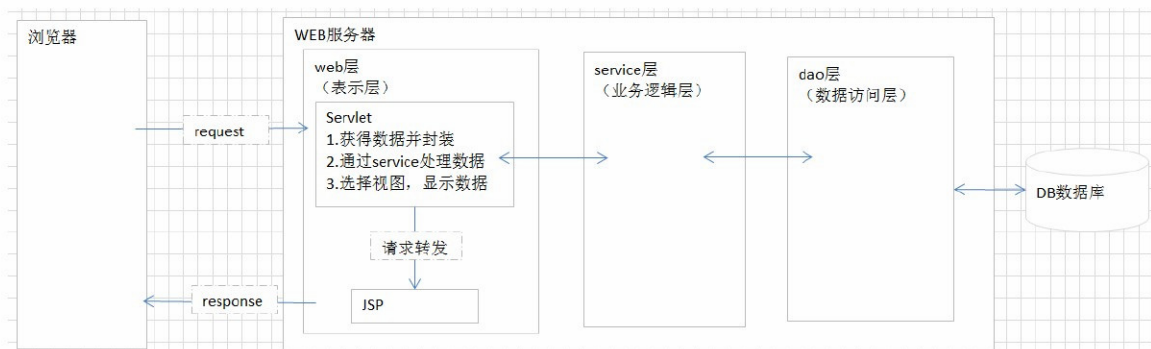


1.2.5 三层架构

JSP 模式是理论基础，但实际开发中，我们常将服务器端程序，根据逻辑进行分层。一般比较常见的是分三层，我们称为：经典三层体系架构。三层分别是：表示层、业务逻辑层、数据访问层。

- 表示层：又称为 web 层，与浏览器进行数据交互的。

- 业务逻辑层：又称为 service 层，专门用于处理业务数据的。
- 数据访问层：又称为 dao 层，与数据库进行数据交换的。将数据库的一条记录与 JavaBean 进行对应。



- 三层架构包命名:

■ 简单版

cn.itcast	公司域名倒写
cn.itcast.dao	dao 层实现
cn.itcast.service	service 层实现
cn.itcast.web.servlet	web 层实现
cn.itcast.domain	JavaBean
cn.itcast.utils	工具

■ 完整版

cn.itcast	公司域名倒写
cn.itcast.xxx	项目名称
cn.itcast.xxx.yyy	子模块
cn.itcast.xxx.yyy.dao	子模块 dao 层接口
cn.itcast.xxx.yyy.dao.impl	子模块 dao 层实现类
cn.itcast.xxx.yyy.service	子模块 service 层接口
cn.itcast.xxx.yyy.service.impl	子模块 service 层实现类
cn.itcast.xxx.yyy.domain	子模块 JavaBean (子模块 yyy 可省略)
cn.itcast.xxx.yyy.web.servlet	子模块 web 层, servlet
cn.itcast.xxx.yyy.web.filter	子模块 web 层, filter
cn.itcast.xxx.utils	工具
cn.itcast.xxx.exception	自定义异常
cn.itcast.xxx.constant	常量

1.3 案例实现

- 重写“day17 查询商品详情”案例，此处只需要修改 jsp 页面的显示。
- 修改 JSP 页面


```
page.jsp product_list.jsp
4 pageEncoding= UTF-8
5 <!--导入核心标签库 -->
6 <%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

```
<!--导入核心标签库 -->
```

```
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

```
page.jsp product_list.jsp
103
104 <c:forEach items="${requestScope.allProduct}" var="p">
105
106 <div class="col-md-2">
107 <a href="product_info.htm">
108 
109 </a>
110 <p><a href="product_info.html" style='color:green'>${p.pname}</a></p>
111 <p><font color="#FF0000">商城价: &yen;${p.shop_price}</font></p>
112 </div>
113
114 </c:forEach>
```

```
<!-- 列表 start -->
```

```
<c:forEach items="${requestScope.allProduct}" var="p">
```

```
<div class="col-md-2">
```

```
<a href="product_info.htm">
```

```

```

```
</a>
```

```
<p><a href="product_info.html" style='color:green'>${p.pname}</a></p>
```

```
<p><font color="#FF0000">商城价: &yen;${p.shop_price}</font></p>
```

```
</div>
```

```
</c:forEach>
```

```
<!-- 列表 end -->
```

第2章 总结

